

דרישות לפרויקט גמר (צד לקוח - React)

1. סקירה כללית

בניית אפליקציית web מלאה ב React

האפליקציה תתקשר עם שרת #C דרך RESTful APIs. ע"י שימוש בספריה axios

הפרויקט ייתן מענה לבעיה קיימת או יספק שירות למשתמש (לדוגמא: ניהול אירועים, ניהול משימות, ניהול דירות/שוכרים, ניהול תלמידים/מורים, מעקב בריאות/כושר, מערכת הזמנות, אתר שיתופי וכו').

2. דרישות פונקציונליות

א. אימות והרשאות משתמש

מימוש הרשמה, התחברות ויציאה מהמערכת.

שימוש באימות מבוסס token (JWT או דומה).

לפחות שני סוגי משתמשים (לדוגמא: משתמש רגיל ומנהל) עם הרשאות שונות.

אבטוח מידע, עמודים ופעולות לפי הרשאות המשתמש.

יש לשמור את מצב ההתחברות של המשתמש כך שלא יצטרך להתחבר מחדש בכל ריענון דף.

ב. פעולות CRUD

לכל ישות עיקרית במערכת (כל סוג מידע מרכזי), יש לאפשר למשתמש לבצע את כל פעולות ה CRUD: הוספה, צפייה, עדכון ומחיקה.

כלומר, כל מידע שנשמר או מוצג במערכת – המשתמש יוכל להוסיף, לערוך ולמחוק אותו בקלות דרך הממשק.

ג. שלילת נתונים וניהול מצב (State Management)

יש לשלוף נתונים מהשרת #C באמצעות קריאות שרת ולהציג אותם באפליקציה (לדוגמא: רשימת משימות, פרטי משתמשים).

לאחר שלילת הנתונים, יש לשמור ולנהל אותם בעזרת Redux – כך שכל שינוי במידע (הוספה, עדכון, מחיקה וכו') יתעדכן אוטומטית בכל חלקי האפליקציה.

יש להקפיד שכל המידע המוצג למשתמש יהיה תמיד עדכני, ושהשינויים יתבצעו בצורה מסודרת דרך Redux.

ד. עיצוב וחווית משתמש

האתר צריך להיות מעוצב בצורה נעימה לעין, ועם סידור נכון של האלמנטים.

חשוב שהממשק יהיה נוח לשימוש, ברור, וקל להתמצאות –

כך שכל משתמש יוכל להבין בקלות מה לעשות ואיך להגיע לכל חלק באתר.

ניתן להשתמש בספריות עיצוב (לדוגמא: Material-UI, Ant Design, React-Bootstrap, Tailwind) או CSS מותאם אישית.

ה. ניתוב (Routing)

שימוש בספריית react-router לניווט בין דפי האפליקציה.

לפחות ארבעה דפים עיקריים (למשל: לוח בקרה, פרופיל, פאנל ניהול, הגדרות, היסטוריית פעולות).

ו. טיפול בשגיאות, בדיקת תקינות ומשוב לפעולות המשתמש

הצגת הודעות שגיאה ברורות למשתמש כאשר מתרחשת שגיאה ב API.

בטפסים, יש לבדוק את תקינות הנתונים (ולידציה), ולהציג הודעות שגיאה מתאימות.

הצגת אינדיקטור טעינה בזמן שליפת נתונים.

יש לספק למשתמש משוב ברור על פעולות שביצע, כדי שיבין מה קורה בכל שלב.

3. דרישות טכניות

שימוש ב Hooks | Functional Components של React.

ארגון קוד נכון, מבנה תיקיות וקבצים הגיוני ומסודר, חלוקה נכונה של רכיבים (קומפוננטות) - ניתנים לשימוש חוזר.

הקפדה על קוד קריא ומובן וקל לתחזוקה (שמות, הערות וכו').

שימוש ב TypeScript.

שימוש ב Git לניהול גרסאות ותחזוקת היסטוריית קומיטים מסודרת (מומלץ)

בהצלחה רבה!